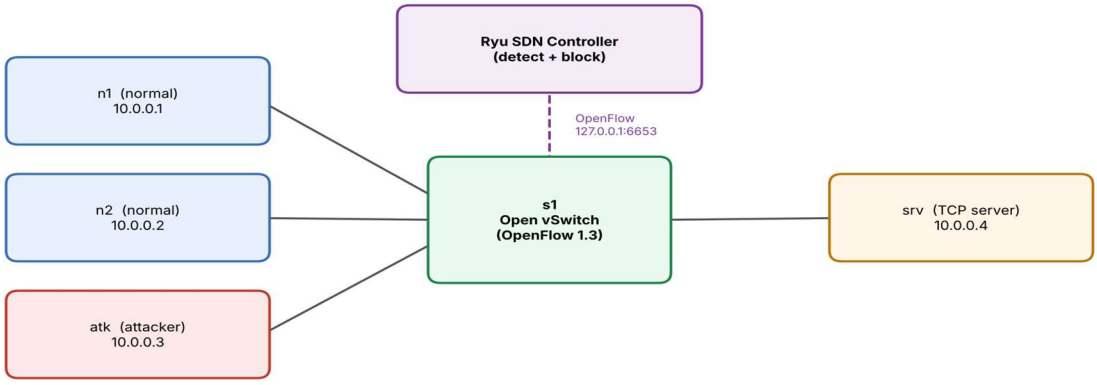
Computer Networks Mini Project
Dynamic Host Blocking System using SDN – Design Document (D1)

Team Details:

- **Abhinav Akinepalli** - **PES1UG25CS013** - Ryu controller (`controller/`), integration testing, test cases
- **Deeksha A** – **PES1UG26CS813** - socket application (`app/`), README, architecture diagram
- **Adithyan Subramanian** - **PES1UG25CS028** - Mininet topology (`topology/`)

1. Problem Understanding and Architecture

Problem & goal. One host flooding a server with new connections can overwhelm it. Using SDN, we detect such a host inside the network and block it automatically while normal hosts keep working. The system has three parts: a socket application (server and clients), a Mininet topology, and a Ryu SDN controller that detects and blocks.



Dynamic Host Blocking System — Architecture

New connections (TCP SYN) reach the controller; >20 in 5 s from one host triggers a priority-100 drop rule.

Packet flow. The first packet of a new connection has no matching flow, so the switch sends it to the controller. The controller counts it per source host and either installs a per-connection flow and forwards it, or, if the host is over the threshold, installs a drop rule. Later packets are forwarded by the switch alone.

2. TCP Socket Implementation

- **Server** (`app/server.py`): multithreaded TCP server, one thread per client; logs each connection (IP, port, time).
- **Normal client** (`app/client.py`): sends messages at a steady, low rate.
- **Attacker client** (`app/attacker.py`): opens connections at the rate set by `--rate`, emulating a flood.

- **Design decisions:** threads stop one slow client blocking others; TCP makes every new connection a visible SYN for the controller to count; the attacker rate is a parameter for testing.

3. Mininet + SDN Implementation

Topology. Open vSwitch `s1`, server `srv` (10.0.0.4), normal hosts `n1 (10.0.0.1)`, `n2 (10.0.0.2)` and attacker `atk(10.0.0.3)`, connected to a remote controller at 127.0.0.1:6653 over OpenFlow 1.3.

Controller (`controller/dhbs_controller.py`, Ryu):

- **Learning switch with per-connection flows:** exact match on IPs, protocol and ports; 10 s idle timeout.
- **Detection:** a new connection is a TCP SYN without ACK (or the first packet of a TCP flow); a sliding window per source IP flags more than 20 in 5 s.
- **Mitigation:** a priority-100 drop rule for the host on all switches, expiring after 60 s; events (`NEW_CONN`, `BLOCK`, `UNBLOCK`) go to `logs/controller_events.log`.

Design decisions – Exact-match flows make every new connection reach the controller; a per-host flow would hide the flood. The drop rule (priority 100) outranks forwarding flows (priority 10), so it also cuts existing attacker flows, and the timed block lets the host recover. A configurable whitelist protects trusted hosts; threshold, window and block time are tunable via DHBS_THRESHOLD, DHBS_WINDOW, DHBS_BLOCK_SECONDS.

4. Socket–SDN Integration

The socket application runs unmodified on the Mininet hosts; the controller only observes its traffic at the switch. The attacker's rate (50 connections/s) exceeds the threshold (20 per 5 s) and triggers the block, while normal clients (1 connections every 2 sec) stay below it.

5. Initial Testing

#	Test	Expected result	Result
1	Normal host reaches server	Succeeds; per-connection flows installed	Pass
2	Attacker floods server	`BLOCK` logged; priority-100 drop rule installed	Pass
3	Normal hosts during attack	Still reach the server	Pass
4	Block expiry	`UNBLOCK` after 60 s; attacker allowed again	Pass

Evidence. Test screenshots (controller BLOCK log, dump-flows drop rule, attacker cut off by ping, normal clients still working) are in the /docs folder.

6. How to Build and Run

Tested on Ubuntu 22.04.3 LTS. Ryu runs in a Python 3.9 virtual environment (it does not build on newer Python). Full setup and run commands are in the repository README.

In brief: (1) start the Ryu controller — ryu-manager controller/dhbs_controller.py; (2) launch the topology — sudo python3 topology/topology.py; (3) in the Mininet CLI run the server and clients — srv python3 app/server.py &, n1/n2 python3 app/client.py --server 10.0.0.4, atk python3 app/attacker.py --server 10.0.0.4 --rate 50. Verify with logs/controller_events.log (look for BLOCK host=10.0.0.3) and sh ovs-ofctl -O OpenFlow13 dump-flows s1. Clean up with sudo mn -c.